Introduction to Python

Python is a high-level, interpreted, and general-purpose programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and allows programmers to express concepts in fewer lines of code compared to languages like C++ or Java.

Key Features of Python

- Easy to Learn & Readable Syntax
- Uses indentation (whitespace) to define blocks of code.
- Minimalistic syntax compared to other languages.
- Interpreted Language
- Executes code line-by-line, making debugging easier.
- No need for explicit compilation (unlike C or Java).
- Cross-Platform Compatibility
- Runs on Windows, macOS, Linux, and other operating systems.
- Dynamically Typed
- No need to declare variable types explicitly.
- Extensive Standard Library
- Includes built-in modules for file I/O, networking, databases, and more.
- Supports Multiple Programming Paradigms
- Procedural (Functions & Modules)
- Object-Oriented (Classes & Objects)
- Functional (Lambda, Map, Filter)
- Large Community & Ecosystem
- Open-source with a vast collection of third-party libraries (e.g., NumPy, Pandas, Django).

Applications of Python

Python is widely used across various domains due to its versatility.

A. Web Development

Frameworks: Django, Flask, FastAPI

Used to build scalable and secure web applications.

B. Data Science & Machine Learning

Libraries: NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, PyTorch

Used for data analysis, visualization, and AI model training.

C. Automation & Scripting

Automates repetitive tasks (e.g., file handling, web scraping with BeautifulSoup, Selenium).

D. Scientific Computing

Used in physics, bioinformatics, and engineering simulations.

E. Game Development

Libraries: Pygame, Panda3D

F. Cybersecurity

Used for penetration testing, network scanning, and malware analysis.

G. Desktop Applications

Frameworks: Tkinter, PyQt, Kivy

H. Internet of Things (IoT)

Used in Raspberry Pi and microcontroller programming.

I. Finance & Trading

Algorithmic trading, risk management, and financial modeling.

Python vs Other Programming Languages

Feature	Python	Java	C++	JavaScript
Syntax	Simple, readable	Verbose	Complex	C-like syntax
Speed	Slower (interpreted)	Faster (JVM)	Very fast	Moderate
Use Case	General-purpose	Enterprise apps	System programming	Web development

Installing Python and Setting Up the Environment on Windows 11

Step 1: Download Python

Visit the official Python website:

Go to https://www.python.org/downloads/

Download the latest stable version:

Click the yellow "Download Python" button (e.g., Python 3.12.x).

Step 2: Install Python

Run the installer:

Locate the downloaded .exe file (e.g., python-3.12.x-amd64.exe) and double-click it.

Enable "Add Python to PATH" (IMPORTANT):

Check the box at the bottom that says "Add python.exe to PATH".

This allows you to run Python from the Command Prompt.

Choose installation options:

Select "Install Now" for a standard installation.

Alternatively, choose "Customize installation" if you need advanced options.

Wait for installation to complete:

Python will install

in C:\Users\<YourUsername>\AppData\Local\Programs\Python\Python312\ (or similar).

Verify installation:

Click "Close" once the installation is done.

Step 3: Verify Python Installation

Open Command Prompt:

Press Win + R, type cmd, and hit Enter.

Check Python version:

Type:

```sh
python --version
```

- If installed correctly, it should display:

```sh
Python 3.12.x
```

3. **Check Pip (Python package manager)**:
   - Type:

```sh
pip --version
```

   - It should show the pip version.

Step 4: Set Up a Virtual Environment (Recommended)

Virtual environments help manage dependencies for different projects.

1. **Install `virtualenv` (if not installed)**:
   - Run:

```sh
pip install virtualenv
```

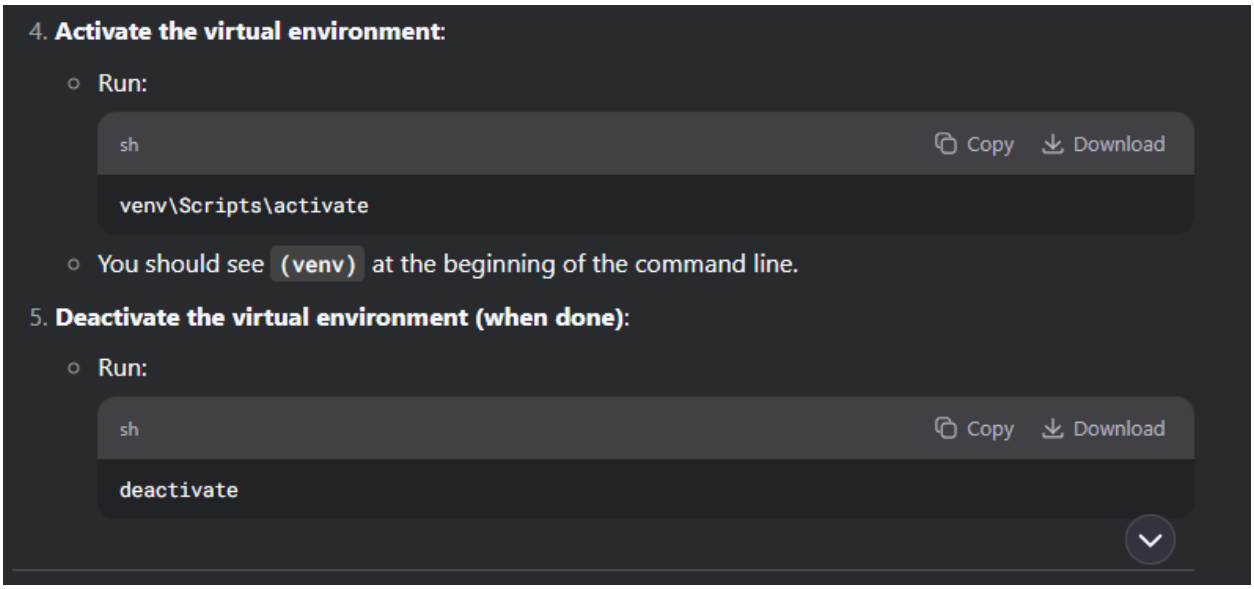
2. **Create a project folder**:
   - Open Command Prompt and navigate to your desired directory:

```sh
cd C:\Users\<YourUsername>\Projects
mkdir my_python_project
cd my_python_project
```

3. **Create a virtual environment**:
   - Run:

```sh
python -m venv venv
```

   - This creates a `venv` folder containing the isolated Python environment.



Step 4: Install an IDE

PyCharm (For Advanced Users)

Download PyCharm Community Edition.

Install and configure Python interpreter in settings.

Python Scripts and Interactive Mode

Introduction to Python Execution Modes

Python can be run in two primary modes:

Interactive Mode

Script Mode

Understanding both modes is essential for effective Python development, debugging, and automation.

Interactive Mode

✅ Definition:

Interactive mode is a method of executing Python code line by line in a Python interpreter shell or REPL (Read-Eval-Print Loop).

✅ How to Start:

Open a terminal or command prompt.

Type python or python3 depending on your system.

The prompt >>> indicates you're in interactive mode.

✅ Features:

Immediate execution of commands.

Ideal for testing small code snippets or performing quick calculations.

Comes with built-in features like auto-completion and documentation access using help().

✅ Example:

python Copy Edit

```
>>> x = 5
>>> x * 2
10
```

✅ Advantages:

Fast feedback: Great for beginners to test and learn.

Debugging aid: Useful for checking functions and data types.

No file needed: Code doesn’t need to be saved in a file.

✅ Disadvantages:

Temporary: Code is not saved unless explicitly written to a file.

Limited use: Not ideal for writing large programs or scripts.

Python Scripts

✅ Definition:

A Python script is a file containing Python code, typically saved with the .py extension. It is executed all at once, as opposed to line-by-line.

✅ How to Create:

Use any text editor or IDE (e.g., VS Code, PyCharm, IDLE).

Save the file with a .py extension.

Run it via the terminal or an IDE:



✅ Features:

Structured and reusable code.

Can contain functions, classes, and modules.

Suitable for building complex programs.

✅ Advantages:

Reusable and organized: Code is easy to maintain and scale.

Persistent: Code is stored and can be reused.

Debuggable: Works well with IDEs and debuggers.

✅ Disadvantages:

Slower testing for small code (requires saving and running).

Less immediate than interactive mode.

Comparing Interactive Mode vs Script Mode

Feature	Interactive Mode	Script Mode
Execution	Line-by-line	Whole file at once
Use Case	Testing, learning	Full programs, automation
Persistence	Temporary	Saved in .py files
Performance	Fast for small tasks	Better for large programs
Tooling Support	Basic	Full IDE and tools support

Variables and Data Types in Python

✅ What is a Variable?

A variable is a name used to store a value in memory. Python automatically determines the data type based on the value assigned to the variable. For example:

x = 10 assigns an integer to x.

y = 3.14 assigns a float.

name = "John" assigns a string.

---

1. 🔢 Numeric Data Types

a. Integer (int)

Represents whole numbers.

Examples: 10, -5, 0.

b. Float (float)

Represents numbers with decimal points.

Examples: 3.14, -0.75, 0.0.

c. Complex (complex)

Represents numbers with a real and an imaginary part.

Example: 2 + 3j, where 2 is the real part and 3 is the imaginary part.

You can access the real and imaginary parts using .real and .imag.

---

2. 📝 String Data Type and Manipulation

a. Defining Strings

Strings can be enclosed in single ('Hello'), double ("Hello"), or triple quotes ('''Hello''' or """Hello""") for multi-line strings.

b. String Operations

Concatenation: Combining strings using +. Example: "Hello" + " World" gives "Hello World".

Repetition: Using * to repeat a string. Example: "Hi" * 3 gives "HiHiHi".

Indexing: Accessing characters by position. Example: "Python"[0] gives 'P'.

Slicing: Extracting a portion of the string. Example: "Python"[1:4] gives 'yth'.

Length: The len() function returns the number of characters. Example: len("Python") is 6.

c. Common String Methods

.upper() converts all letters to uppercase.

.lower() converts to lowercase.

.capitalize() capitalizes the first character.

.replace(old, new) replaces a substring.

.split() splits a string into a list.

"substring" in string checks for presence of a substring.

---

3. 🧺 **Tuples**

A tuple is an ordered collection of items that cannot be changed (immutable). Tuples are created using parentheses, like (1, 2, 3).

Tuples can contain elements of different data types, e.g., ("apple", 3.14, 7).

Elements are accessed using indexing.

Tuple unpacking allows assigning each element to a variable: a, b, c = (1, 2, 3).

---

4. 📦 **Dictionaries**

A dictionary is a collection of key-value pairs, created using curly braces: {"key": "value"}.

Example: {"name": "Alice", "age": 22, "major": "CS"}.

Values are accessed by keys, such as student["name"] returning "Alice".

You can add new key-value pairs or update existing ones.

Common dictionary methods:

.keys() returns all keys.

.values() returns all values.

.items() returns key-value pairs.

---

5. ✅ **Boolean Values and Operators**

a. Boolean Values

Python has two Boolean values: True and False.

b. Comparison Operators

== checks if values are equal.

!= checks if values are not equal.

> checks if left is greater than right.

< checks if left is less than right.

>= and <= check for greater than or equal, and less than or equal respectively.

c. Logical Operators

and returns True if both operands are true.

or returns True if at least one operand is true.

not inverts the Boolean value.

Control Flow in Python

Control flow in Python refers to the order in which individual statements, instructions, or function calls are executed or evaluated. Python provides several control flow tools to help programmers write logic for decision-making, looping, and exception handling.

1. Conditional Statements (Decision-Making)

if **Statement**

Used to execute a block of code only if a condition is true.

```
x = 10
if x > 5:
    print("x is greater than 5")
```

if-else **Statement**

Provides an alternative block when the condition is false.

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is 5 or less")
```

if-elif-else **Statement**

Used for multiple conditions.

```
x = 7
if x > 10:
    print("x is greater than 10")
elif x == 7:
    print("x is 7")
else:
    print("x is less than 10 and not 7")
```

2. Looping Statements

Looping allows the execution of a block of code repeatedly.

while **Loop**

Repeats as long as the condition is true.

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

for **Loop**

Iterates over a sequence (like a list, tuple, string, or range).

```
for i in range(1, 6):
    print(i)
```

Iterating over other sequences:

```
for char in "Python":
    print(char)
```

3. Loop Control Statements

Used to control the execution of loops.

break

Exits the current loop prematurely.

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

continue

Skips the current iteration and continues with the next.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

pass

Does nothing. Acts as a placeholder.

```
for i in range(5):
    pass  # Placeholder for future code
```

4. The else **Clause in Loops**

You can attach an else block to a loop. It runs only if the loop wasn't terminated by break.

```
for i in range(5):
    print(i)
else:
    print("Loop completed successfully")
```

5. Match-Case Statement (Python 3.10+)

Similar to switch-case in other languages.

```
status = 404
match status:
    case 200:
        print("OK")
    case 404:
        print("Not Found")
    case _:
```

```
    print("Unknown Status")
```

6. Exception Handling (Error Control Flow)

try-except **Block**

Handles exceptions during runtime.

```
try:
   result = 10 / 0
except ZeroDivisionError:
   print("Cannot divide by zero")
```

else **and** finally

else: Runs if no exception occurs.

finally: Always runs, regardless of exceptions.

```
try:
   result = 10 / 2
except ZeroDivisionError:
   print("Error")
else:
   print("No error occurred")
finally:
   print("Execution complete")
```

Functions in Python

Functions are reusable blocks of code that perform a specific task. They help break programs into smaller, manageable, and modular chunks.

1. **Defining Functions and Function Arguments**

➤ Defining a Function

In Python, a function is defined using the def keyword:

```
def greet():
   print("Hello, World!")
```

You call it using:

```
greet()
```

➤ Function Arguments

Functions can take parameters (also called arguments) to work with data:

```
def greet(name):
    print(f"Hello, {name}!")
```

Types of Arguments

Positional Arguments:

Must be passed in the same order as defined.

```
def add(x, y):
    return x + y
add(2, 3)
# Output: 5
```

Default Arguments:

Provide default values.

```
def greet(name="Guest"):
    print(f"Hello, {name}!")
```

Keyword Arguments:

Passed with parameter names.

```
greet(name="Alice")
```

Variable-length Arguments:

*args: Non-keyworded variable-length arguments (tuple)

**kwargs: Keyworded variable-length arguments (dictionary)

```
def show(*args, **kwargs):
    print(args)
    print(kwargs)
show(1, 2, 3, a=4, b=5)
```

2. **Return Values and Function Scopes**

➤ Return Values

Functions can return values using the return statement:

```
def square(x):
```

```
    return x * x
```

You can also return multiple values (as a tuple):

```
def operations(a, b):
    return a + b, a - b
add, sub = operations(10, 5)
```

➤ Function Scope

Variables in Python have different **scopes**:

Local Scope – Inside the function.

Global Scope – Outside all functions.

Nonlocal Scope – Used in nested functions.

```
x = "global"
def outer():
    x = "outer"
    def inner():
        nonlocal x
        x = "inner"
    inner()
    print(x)
outer()
# Output: inner
```

To modify global variables:

```
x = 5
def change():
    global x
    x = 10
```

3. **Lambda Functions and Higher-Order Functions**

➤ **Lambda Functions**

A lambda function in Python is a small, anonymous function defined with the lambda keyword.

Unlike regular functions defined with def, lambda functions are:

Anonymous (no name)

Typically used for short, simple operations

Limited to a single expression

Syntax:

lambda arguments: expression

Examples

Simple lambda function

```
double = lambda x: x * 2
print(double(5))  # Output: 10
```

Multiple arguments

```
add = lambda a, b: a + b
print(add(3, 4))  # Output: 7
```

Using with built-in functions

Functions that take other functions as arguments or return them.

map()

Applies a function to each element in an iterable:

filter()

Filters elements based on a condition:

```
# With map()
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared)  # Output: [1, 4, 9, 16]


# With filter()
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # Output: [2, 4]
```

Sorting with lambda

```
people = [{'name': 'Alice', 'age': 25}, {'name': 'Bob', 'age': 20}]
people_sorted = sorted(people, key=lambda person: person['age'])
```

```
print(people_sorted)
# Output: [{'name': 'Bob', 'age': 20}, {'name': 'Alice', 'age': 25}]
```

When to Use Lambda Functions

For short, simple operations that are used only once

As arguments to higher-order functions like map(), filter(), sorted()

When you need a quick function without formally defining it with def

Limitations

Can only contain a single expression

No statements or multiple expressions allowed

No annotations or documentation strings

Less readable for complex operations (in such cases, use regular functions)

4. **Recursion**

Recursion is a programming technique where a function calls itself in order to solve a problem.

It's particularly useful for problems that can be broken down into smaller, similar subproblems.

Basic Structure of a Recursive Function

Every recursive function needs:

A base case - the condition that stops the recursion

A recursive case - where the function calls itself with a modified input

```
def recursive_function(parameters):
    # Base case
    if base_case_condition:
        return base_case_value
    # Recursive case
    else:
        return recursive_function(modified_parameters)
```

Example 1: Factorial Calculation

```
def factorial(n):
    # Base case: 0! = 1 or 1! = 1
    if n == 0 or n == 1:
```

```
    return 1
  # Recursive case: n! = n * (n-1)!
  else:
    return n * factorial(n - 1)
print(factorial(5))  # Output: 120
```

Example 2: Fibonacci Sequence

```
def fibonacci(n):
  # Base cases
  if n == 0:
    return 0
  elif n == 1:
    return 1
  # Recursive case: fib(n) = fib(n-1) + fib(n-2)
  else:
    return fibonacci(n - 1) + fibonacci(n - 2)
print(fibonacci(7))  # Output: 13
```

Example 3: Directory Tree Traversal

Directory tree traversal means visiting all files and subdirectories inside a given directory. This is useful for tasks like:

Listing all files in a folder

Searching for specific files

Calculating folder sizes

Processing multiple files at once

1. Using os.listdir() (Simple Recursive Approach)

This method lists all files and subdirectories recursively.

Code Example

```
import os
```

```
# List all files and directories in the current directory
contents = os.listdir()
print(contents)
# List contents of a specific directory
contents = os.listdir('D:/2')
print(contents)
```

Using os.walk() in Python

The os.walk() function is a powerful tool for traversing directory trees in Python. It generates the file names in a directory tree by walking the tree either top-down or bottom-up.

Basic Syntax

```
import os
for root, dirs, files in os.walk(top, topdown=True, onerror=None, followlinks=False):
    # Process files and directories
```

Parameters

- top: The root directory to start walking from
- topdown: If True (default), walk top-down (parent before children). If False, walk bottom-up.
- onerror: Optional function to handle errors
- followlinks: If True, follow symbolic links (default is False)

Basic Example

```
import os
# Walk through all files and directories starting from the current directory
for root, dirs, files in os.walk('.'):
    print(f"Current directory: {root}")
    print(f"Subdirectories: {dirs}")
    print(f"Files: {files}")
    print("-" * 40)
```

Common Use Cases

1. Find all files with a specific extension

```
import os
py_files = []
for root, dirs, files in os.walk('.'):
    for file in files:
        if file.endswith('.py'):
            py_files.append(os.path.join(root, file))
print("Python files found:")
for file in py_files:
    print(file)
```

2. Calculate total size of all files in a directory

```
import os
total_size = 0
for root, dirs, files in os.walk('.'):
    for file in files:
        file_path = os.path.join(root, file)
        total_size += os.path.getsize(file_path)
print(f"Total size: {total_size} bytes")
```

3. Find empty directories

```
import os
empty_dirs = []
for root, dirs, files in os.walk('.'):
    if not dirs and not files:
        empty_dirs.append(root)
print("Empty directories:")
for dir in empty_dirs:
    print(dir)
```

4. Modify directory tree while walking (top-down only)

```
import os
# Skip directories named 'temp'
```

```
for root, dirs, files in os.walk('.', topdown=True):
    if 'temp' in dirs:
        dirs.remove('temp')  # This prevents os.walk from descending into 'temp'

    # Process remaining files and directories
    print(f"Processing {root}")
```

Important Notes

1. os.walk() returns a generator, not a list, which makes it memory efficient for large directory trees.
2. The dirs list can be modified in-place to control which directories are visited (top-down only).
3. Be careful with symbolic links - by default they're not followed to avoid infinite loops.
4. For case-sensitive filesystems, all names are returned exactly as stored by the filesystem.

File Handling: Reading and writing text files

File handling in Python is a common and essential operation, especially for tasks involving reading from or writing to text files. Python provides built-in functions and methods to handle files effectively. Here's a quick guide on **reading** and **writing** text files in Python.

Opening a File

Python uses the built-in open() function to open files.

```
file = open("filename.txt", "mode")
```

"r" – Read (default)

"w" – Write (overwrites file if exists)

"a" – Append (adds to the end)

"x" – Create (fails if file exists)

"t" – Text mode (default)

"b" – Binary mode

Reading from a File

1. Read entire file

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

2. Read line by line

```
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())  # Removes trailing newline
```

3. Read specific number of characters

```
with open("example.txt", "r") as file:
    content = file.read(10)  # Reads first 10 characters
    print(content)
```

Writing to a File

1. Overwrite (write mode "w")

```
with open("example.txt", "w") as file:
    file.write("Hello, world!")
```

2. Append to a file

```
with open("example.txt", "a") as file:
    file.write("\nAppended line.")
```

Other Useful Methods

- readline() – Reads one line at a time.
- readlines() – Returns a list of lines.
- writelines(list) – Writes a list of strings to the file.

Best Practice: Using with Statement

Using with is the recommended way to handle files because it automatically closes the file after the block is executed, even if exceptions occur.

Working with CSV and JSON files in python

Working with **CSV** and **JSON** files in Python is straightforward using built-in libraries: csv for CSV files and json for JSON files. Here's a guide to help you get started with reading and writing both formats.

- **CSV Files**

1. Reading a CSV File

```
import csv
with open('data.csv', newline=' ') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)
```

2. Writing to a CSV File

```
import csv
data = [
    ['Name', 'Age', 'City'],
    ['Alice', 30, 'New York'],
    ['Bob', 25, 'London']
]

with open('output.csv', mode='w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(data)
```

3. Using DictReader and DictWriter

```
# Reading
with open('data.csv', newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        print(row['Name'], row['Age'])

# Writing
fieldnames = ['Name', 'Age', 'City']
with open('output.csv', mode='w', newline='') as csvfile:
```

```
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerow({'Name': 'Alice', 'Age': 30, 'City': 'New York'})
```

🔹 JSON Files

1. Reading a JSON File

```
import json
with open('data.json', 'r') as jsonfile:
    data = json.load(jsonfile)
    print(data)
```

2. Writing to a JSON File

```
import json
data = {
    'name': 'Alice',
    'age': 30,
    'city': 'New York'
}

with open('output.json', 'w') as jsonfile:
    json.dump(data, jsonfile, indent=4)
```

🔄 Converting Between CSV and JSON

CSV → JSON

```
import csv, json
csv_file = 'data.csv'
json_file = 'data.json'
with open(csv_file, newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    rows = list(reader)
```

```
with open(json_file, 'w') as jsonfile:
    json.dump(rows, jsonfile, indent=4)
```

JSON → CSV

```
import json, csv
json_file = 'data.json'
csv_file = 'data.csv'
with open(json_file) as jsonfile:
    data = json.load(jsonfile)
with open(csv_file, 'w', newline='') as csvfile:
    writer = csv.DictWriter(csvfile, fieldnames=data[0].keys())
    writer.writeheader()
    writer.writerows(data)
```